

Current-generation MLFF campaign CLI specification

Version: mdstats 0.20.242a0 current contract Status: implemented for the P1-P5 functional campaign lifecycle

Purpose and authority

The campaign CLI projects the current source, statistical, target-size, post-selection, and production owners into one restartable user workflow. It does not replace those owners or create a second scientific state machine. Architecture defines ownership and data flow; this specification defines the public command, configuration, persistence, currentness, and failure contract.

The source-checkout entry point is:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml <command>
```

Public command surface

The parser exposes exactly these top-level commands:

```
init
doctor
prepare
select-target-size
cross-validate
train-production
storage {report,cleanup,archive {create,list,verify,restore,reclaim},deduplicate}
qualification {status,run,activate-locked}
status
advance
guide
```

The scientific lifecycle is:

```
init -> doctor -> prepare -> select-target-size -> cross-validate -> train-production
```

storage is orthogonal artifact management. status and advance derive their projection from the same current owners. The current campaign has no separate pre-screen gate. qualification is the separate downstream post-production lifecycle (P7, mlff_p7_post_production_qualification_spec.md) over the exact frozen final publication: status is observational, run executes or resumes the nonlocked components, and activate-locked is the only, never-automatic path to the reserved locked test. It is not a training-lifecycle stage and cannot feed back into any P1-P6 authority.

The downstream design is an ordered collection of distinct qualified target sizes, so select-target-size, cross-validate and train-production each carry a size dimension over one shared prepared generation. That dimension adds no second campaign, no per-size subcampaign, and no cross-size winner rule.

No command may silently skip a failed, stale, waiting, or incompatible current authority. An automatic target-size diagnostic that completes without a recommendation is reported as a successful diagnostic result: it leaves the current provisional design unchanged and does not prohibit an explicit target-size choice.

User-visible layout

The operator supplies campaign.toml. The workspace contains:

```
<workspace>/campaign-manifest.json
<workspace>/mdstats/campaign.sqlite3
<workspace>/mdstats/                # current records and reconstructible caches
<workspace>/data/                   # current MACE materializations
<workspace>/runs/                   # authorized checkpoints and logs
```

```

<workspace>/models/          # current production publication
<workspace>/results/         # bounded summaries and cleanup reports

```

SQLite rows and content-addressed files are authoritative only through their own current owners. File existence, a caller-held object, or a cache path is not evidence of currentness.

Configuration contract

campaign.toml owns requested policy and source locations. Runtime observations and durable scientific outcomes are persisted separately. Paths are resolved relative to the configuration file, and a non-positive execution timeout means no campaign wall-clock timeout.

The generated current configuration exposes:

```

[target_data.size_convergence]
target_size_power_min = 7
target_size_power_max = 14
evaluation_size_powers = [8, 9, 10]
fidelity_epochs = [1, 3, 10]

[post_selection.cv]
fold_count = 3
partition_seed = 104729
seeds = [0]
max_num_epochs = 30
purge_components_between_roles = 0
checkpoint_maximum_target_force_rmse_ev_per_angstrom = 0.045
acceptance_metric = "target_force_rmse_ev_per_angstrom"
acceptance_maximum = 0.045

[post_selection.production]
seeds = [1]
committee_policy = "all_qualified_final_seeds"

```

The generated campaign is a foundation (multihead_replay) campaign, so it exposes both foundation-CV defaults: checkpoint_maximum_target_force_rmse_ev_per_angstrom = 0.045 (the CV checkpoint target-force competence ceiling on M_mon) and acceptance_maximum = 0.045 (the held-out fold acceptance ceiling, in units of acceptance_metric). Fresh production checkpoints use [acceptance].maximum_target_force_rmse_ev_per_angstrom (generated 0.030). All three post-selection thresholds are independently configurable policy parameters. When omitted, checkpoint_maximum_target_force_rmse_ev_per_angstrom defaults to 0.045 for foundation modes; acceptance_maximum defaults to 0.045 for foundation modes and 0.030 for scratch; and production checkpoint quality defaults to 0.030. Scratch CV checkpoints keep the [acceptance] ceiling. An explicit value is never rewritten, and its units follow acceptance_metric for acceptance_maximum and target-force RMSE (eV/angstrom) for checkpoint ceilings. The exact resolution table is owned by the P5 specification section 12.1.

The configured power ceiling is not a fixed scientific constant. Candidates are additionally bounded by the available P_train population and the current policy. Optimizer seeds are authored by the sole enabled training method; there is no separate target-size seed namespace. Pre-target fold-count and partition-seed fields are not current target-size/CV authoring controls.

The learned model precision is single (FP32) or double (FP64). Critical mdstats reductions, geometry/statistical arithmetic, and persistent bookkeeping remain FP64. The acceleration backend, MACE interface, and runtime identity are bound by the doctor/current protocol record.

Policy configuration is fail-closed at the existing semantic owners. Finite real fields reject booleans and strings, integer fields require actual integers, boolean fields require actual booleans, and collection elements are validated before canonical ordering or float conversion. This includes the target-size normalization reference, global objective, and configuration-weight policies. Current-schema readers reject malformed values before identity or execution; only an explicitly supported historical reader may apply a historical representation.

Command behavior

init

init writes an annotated configuration with current target-size and post-selection sections. It must not generate retired lifecycle sections or imply an unconfigured target-size ceiling. Existing configuration files are not over-written without the explicit force behavior defined by the parser.

doctor

doctor checks source paths, manifest inputs, foundation/replay identity, package/runtime imports, precision wrappers, requested backend, and available resources. A successful result records the exact runtime realization used by later owners. Unsupported hardware or unavailable long-production resources remain an explicit unavailable/deferred condition, never a fabricated pass.

For the current single-source replay interface doctor validates prerequisites only. It must not call the replay-wide prediction-cache builder, perform replay pseudo inference, run prediction-dependent pseudo qualification, create mode-specific train/monitor materializations, or publish any current single-source replay alias. Its report states that prediction-dependent eligibility, cardinality, and qualification are deferred to prepare, and `replay_plan_doctor` is retired from current single-source use rather than reused with weaker semantics. Supported legacy split-file replay keeps its existing doctor-owned realized qualification, which is realizable under that contract without constructing anything.

prepare

prepare authenticates the manifest and constructs/reuses the current neutral substrate:

```
source/frame/label authority
-> protected statistical relations
-> one P_train/M3 split and pi_train/pi_eval
-> one common target-size preparation
```

It selects no target size, trains no candidate, ranks no checkpoint, and publishes no final model. It is restartable and idempotent when all current inputs and identities match, including when the current generation already carries a complete automatic diagnostic: an unchanged prepare is a successful no-op that leaves the canonical generation, its diagnostic evidence, and its derived result view exactly as they were.

prepare also owns replay construction and reuse. Before any expensive work it runs the cheap canonical replay configuration/topology validation, so a conflicting or unsupported label selector, an inexact split domain, a mixed replay interface, or a replay declaration incompatible with the resolved training method is rejected at entry. `TRUE_DFT` preparation authenticates the source/index/true-label basis, builds or reuses the deterministic split, materializes and authenticates the required transports, and performs zero foundation inference. Explicit pseudo preparation additionally resolves the current foundation prediction policy, builds or reuses the prediction cache under a same-key single-flight fence, qualifies the cached predictions, materializes the pseudo training transport and the independent `TRUE_DFT` monitor, enforces the post-qualification minimum counts, and retires its accelerator owner before returning. It then publishes exactly one interface- and mode-appropriate current replay alias set - including realized replay qualification - in one transaction, retiring every inapplicable alias.

prepare is COMPLETE only when every configured preparation prerequisite for that invocation is current and authenticated. A replay failure after the target-size generation is published makes public prepare incomplete without rolling back the independently valid target-size generation.

prepare is the sole command permitted to interpret live inputs, so it also owns detecting that they changed. Before reusing the stored lower-level catalog it compares the approved manifest digest that catalog was built from and each source's own byte-identity and control signatures against the files on disk. Materially changed valid inputs are routed through the ordinary catalog reconstruction path and committed as a fresh canonical generation; malformed or unapproved changes still fail under their existing owners. No operator flag is required for this. --approve-manifest records the reviewed digest; --refresh-inferences refreshes proposed source metadata before approval; --rebuild-catalog remains an explicit unconditional reconstruction request, not the mechanism by which ordinary input changes are noticed.

Durable publication is create-or-verify. A prepared component, generation manifest, or normalized frame entry whose content identity already exists on disk is authenticated against that identity – for a frame entry, including every array member's hash, shape, and dtype – before this generation may reuse it. Conflicting or corrupt content fails before adoption and is never overwritten, because another adopted generation may still depend on the bytes that are actually there.

Adoption is compare-and-set against the currentness token captured *before* the expensive construction began, and no campaign-wide writer lock is held across that construction. Two concurrent preparations that produce the same prepared identity converge on one generation; a preparation whose snapshot was built against superseded state fails closed rather than advancing the newer generation.

At the destructive generation boundary, obsolete derived target-size records are detected before semantic decoding, quarantined under a namespace no current loader reads, and rejected rather than migrated. Lower-level source or content caches are reusable only after current-owner revalidation.

select-target-size

```
select-target-size <N>
select-target-size --auto
select-target-size --reset
[--horizon-cv <positive integer>] [--horizon <positive integer>]
```

This command owns the **provisional** downstream training design and freezes nothing. A bare invocation is invalid and must give actionable usage guidance; <N>, --auto and --reset are mutually exclusive, and exactly one of them is required.

The provisional design is an **ordered collection of distinct qualified sizes**, not a single choice. One prepared generation is expensive and deliberately reusable, so requesting a second qualified size must add an experiment rather than replace the one already requested:

- a size not yet in the design is **appended**;
- a size already in the design has its **complete entry replaced in place**, keeping its position;
- duplicate N in authoritative state is corruption and is never silently deduplicated;
- the empty collection is the canonical unselected state; no placeholder record exists, and cross-validate refuses zero selected sizes.

select-target-size <N> loads the current prepared generation, establishes currentness, resolves the P2 qualified candidate set, requires N to be one of those configured qualified candidates, derives $T_N = \text{pi_train}[:N]$, validates that membership through the existing P2 training-order owner, resolves a complete per-size horizon snapshot, and CAS-publishes the merged collection. It performs **no** target-size candidate training and no EVAL2 work.

`--horizon-cv` sets the CV max epochs and `--horizon` the final-production max epochs **for the size this invocation touches**. Each successful invocation resolves a complete entry: an omitted flag resolves from `[post_selection.cv].max_num_epochs` (default 30) or `[training].max_num_epochs` (default 30) *at that moment*, and the resolved values are persisted, so later configuration edits do not mutate an entry that already exists and untouched sibling entries never drift. Reselecting an existing size deliberately re-resolves every omitted horizon for the new invocation. An earlier override never becomes a sticky default. The CLI never rewrites `campaign.toml`.

`select-target-size --reset` atomically clears every provisional entry. It is pre-freeze only, performs no numerical work, preserves the prepared generation and any valid automatic-diagnostic evidence and report, and is mutually exclusive with `--horizon-cv` and `--horizon`. Resetting an already-empty design is idempotent and appends no revision.

`select-target-size --auto` runs or reuses the optional automatic diagnostic. It runs the configurable ladder and direct evaluation populations through the authenticated continuation

`n1 / M1 -> n2 / M2 -> n3 / M3`

with paired optimizer seeds from the sole enabled method. Each candidate is an exact prefix of the one `pi_train` order. The reducer publishes either a **recommended** N with the exact identity of `pi_train[:N]`, or a typed no-recommendation outcome. Replay and later validation evidence cannot affect that computation. A valid recommendation becomes the current provisional N.

A completed diagnostic that produces no recommendation must not partially modify any proposal field: the diagnostic evidence and its portable report are committed, any previously valid proposal is unchanged, and the command returns normal success while stating that no recommendation was established.

If a current reusable complete diagnostic already exists for the campaign's automatic-screen scientific/execution identity, `--auto` authenticates and reloads it, performs zero new TRAIN2/EVAL2 work, and says so. Provisional N, either horizon flag, either configured horizon, report presentation, status wording, report path, and selection-source provenance never invalidate a current diagnostic.

A diagnostic may run for hours. It captures the selection state it intends to update; if a newer explicit choice or a cross-validate freeze intervened, the evidence and report are still published but the stale recommendation is not installed, and the command says so.

Every terminal diagnostic writes one portable, self-contained Markdown report under the campaign `results/` tree at a stable per-generation filename. It is a derived, rebuildable projection of authenticated evidence and is never read as authority.

Candidate learning-rate amplitude and EMA decay are normalized against the configured reference size (`[target_data.size_convergence.optimizer_normalization]`) so a larger candidate does not also receive more optimizer progress; epoch counts, batch size, LR shape, and every other optimizer setting stay fixed across candidates.

The configured ladder ceiling is a practical budget limit. When Nmax remains materially superior to every other successful terminal finalist, it is **recommended** and the result carries the non-blocking warning code `nonconverged_at_configured_ceiling`; status, the derived result view, and the portable report state the warning alongside the recommendation. Inside the practical-equivalence band the smaller finalist is still preferred. Genuinely insufficient comparison remains a typed no-recommendation outcome, and no unconfigured intermediate or rescue size is ever synthesized.

A valid recommendation is merged through the **same** unique-by-N collection owner as a manual choice. The automatic path has no collection authority of its own: it never clears, reorders, or replaces a sibling entry the operator chose, and installing it over an existing entry for the same N follows the ordinary replace-in-place rule.

cross-validate

This command is the freeze boundary, and it freezes the **complete collection atomically**. It requires at least one provisional entry, loads and authenticates the one current prepared generation, revalidates every proposed N against the current qualified candidate set, derives each exact $T_N = \text{pi_train}[:N]$, reproduces every membership digest, and publishes the whole ordered frozen design - every membership and both effective role horizons per size - in one CAS transition before any numerical CV work begins. One unauthenticatable member rejects the entire admission: there is no partial freeze, and a requested multi-size experiment is never quietly reduced to the subset that still validated. No automatic-diagnostic execution head or reducer is required for this transition. After the freeze, `select-target-size` in any form must not mutate the design.

It then runs the existing post-selection cross-validation methodology **for every frozen size**, in frozen selection order: the configured $K \geq 2$ post-selection folds inside exactly that size's T_N , its own frozen H_{cv} , protected relations, fold/seed identities, target-only checkpoint choice, and the all-required-fold/all-required-seed acceptance predicate. Campaign cross-validation is accepted only when **every** frozen size is accepted; a rejected size stays visibly rejected and is never dropped from the design. The size dimension adds no new fold construction, acceptance predicate, cross-size reducer, or scheduler: outer iteration over sizes is serial and shares the existing effective resource allocation. Missing, stale, or failed fold evidence blocks final production while leaving the selected authority unchanged.

train-production

Admission is a **collection-wide barrier**. Before any new production job starts, the complete frozen design is authenticated and every selected size must hold current accepted cross-validation ancestry under its own binding and its own H_{cv} . If any size is missing, stale, corrupt, incomplete, or rejected at that boundary, the invocation starts **no** production job for any size and reports every known blocking N. Immutable evidence published by an earlier attempt keeps whatever currentness its own identity earns; the barrier only prevents new admission from turning an all-sizes experiment into a successful subset.

After admission it runs the existing fresh final-production methodology for every frozen size: fresh start from the accepted foundation on that size's complete exact T_N , under **its own** frozen production horizon, its own accepted cross-validation ancestry, the campaign-common target monitor record shared with cross-validation (no target-size M3 parent), production seeds, representative selection and committee policy, publishing one binding-scoped final-production decision per size. A screen or CV checkpoint is never a production parent, and no size may consume another size's membership, horizons, evidence, pointers, or publication. Publication rechecks currentness at commit time and cannot promote work from a superseded generation. Campaign production is complete only when every selected size has its own current final publication.

For a multi-size design, completing production is a **terminal training-experiment state**, not a release: see qualification.

status, advance, and guide

`status` projects the current owner states and reports the next safe action. `advance` dispatches only the next current lifecycle owner. `guide` prints the same six-command scientific lifecycle and current configuration/restart semantics. Neither command writes a second scientific authority.

Every successful selection, automatic install, reset and `status` renders the **complete** design in selection order: before the freeze, each entry's N, H_{cv} , H_{prod} and provenance with Frozen: no; after it, the same ordered list with each size's own cross-validation and production state. An empty design shows no concrete size, the current default horizons, and the requirement to select at least one N before cross-validation. The per-size summary is presentation only: it never ranks sizes, sorts them by performance, flags a winner, or introduces any cross-size reducer.

advance routes as follows: no provisional entries stops at the target-size decision and never invents an N or opts into --auto; a nonempty provisional design routes to cross-validate; a frozen design with any size not accepted keeps cross-validation as the relevant stage and never advances into production; all sizes accepted with any production incomplete routes to train-production; all production complete with $k == 1$ routes to the existing qualification stage; all production complete with $k > 1$ reports a complete, unqualified multi-size training experiment and offers no further consequential command.

Observation is read-only, coherent, and authenticated. One lifecycle answer reads the target-size revision, every current per-size binding derived from it, and every post-selection pointer row of every such binding - plus qualification pointer rows only where qualification is actually authorized - inside a single campaign-store read transaction, so the ancestry it reports is one that existed. Iterating sizes with independently timed authoritative reads is not permitted: pointer publication mutates campaign metadata without moving the target-size revision, and an answer may never combine a pre-publication view of one stage or size with a post-publication view of another. Each compact record a pointer names is loaded through its accepted read-only typed store and must reproduce the identity the pointer named before any of its fields - a cross-validation acceptance, a release verdict - influences the report. A missing, unparseable, or misidentified record is reported as blocked. Observation creates no evidence root, opens no provider, and reconstructs no upstream authority.

qualification

Post-production qualification is a single-product release boundary. For a one-size frozen design its methodology, identity, locked-activation semantics and lifecycle are unchanged.

For a multi-size frozen design ($k > 1$) qualification is **unavailable**, and that is a safety boundary rather than a missing algorithm. Several final publications exist and this revision authorizes no rule for deciding which one is the release product; choosing implicitly - first, last, automatic recommendation, or best metric - would be a release decision made silently, and running one-shot locked evidence across sizes would turn reserved data into a target-size comparison. Therefore:

- `qualification run` and `qualification activate-locked` (and every other consequential qualification entry-point) fail closed **before** any attempt, session, evidence root, external-reference request, or locked-cohort access is created or revealed;
- `qualification status` succeeds observationally, writes nothing, and explains the boundary;
- no release-qualified verdict is set and no qualification completion is synthesized;
- the frozen design is never mutated down to one size.

Qualifying one product from a multi-size experiment requires a separate experiment under a fresh prepared generation with exactly one selected size, or separate explicit design authority for multi-product release selection.

storage

Storage reports and manages only campaign-owned artifacts. Read-only inventory is available through storage report. Cleanup tiers require their documented dry-run/apply behavior; archive create, integrity check, and restore are reversible and independently content-checked. External inputs, current scientific records, selected checkpoints, restart state, and diagnostic evidence remain protected.

Persistence and restart

CampaignStore shall:

1. use one SQLite database for durable campaign state;
2. serialize canonical payloads with content identities under their owners;
3. record operational stage state without treating it as scientific authority;
4. preserve the latest successful evidence before process exit;

5. retain bounded event history;
6. never use the database location itself as scientific identity.

Current preparation and post-selection owners rederive their input identities on every reopen. A matching completed record is reused only after source, policy, parent, payload, and currentness authentication. A missing, corrupt, stale, or incompatible derived cache is rebuilt by its owner; it is not silently accepted or translated.

Each current write uses a generation-neutral prepare/restart identity. A historical generation marker may be inspected only by the reject-only cutover detector and may not authorize semantic reconstruction. The exact accepted current-generation P5A6 workspace must remain reopenable without a pre-load rewrite; P6 compatibility evidence is separate from fresh P6 restart evidence.

Configuration changes have selective invalidation:

```
target/frame/label/order/ladder/fidelity/common-preparation change
    -> new target-size generation and descendants
post-selection CV-only change
    -> CV and final-production descendants only
production-only horizon/runtime change
    -> final-production descendants only
```

Provenance-only presentation changes do not change scientific arithmetic.

Failure contract

The CLI fails closed for unapproved or changed manifests, missing or incompatible source/foundation/replay inputs, invalid labels, unresolved protected relations, unsupported policy generation, stale current pointers, missing required folds/seeds, corrupt checkpoint/companion state, no admissible checkpoint for a final-production run, target-size lineage mismatch, incompatible persisted state, or a currentness race at publication.

A cross-validation fold whose nonempty checkpoint candidates all fail mandatory admissibility is not an execution failure. It is a completed rejected fold verdict with no representative and no held-out outer evaluation, binding its candidate records and the mandatory reasons they failed. Sibling folds and later frozen sizes still complete; `cross-validate` then reports the methodological rejection without a traceback (exit status 2) and final production stays unauthorized. An empty candidate set remains missing evidence.

Replay adds the same fail-closed discipline: conflicting or unsupported replay label selectors, inexact replay split-domain values, mixed replay interfaces, a replay declaration incompatible with the resolved training method, a replay frame that does not reproduce its authenticated canonical geometry identity during a long read, a replay source mutated before prepared aliases become current, a prepared replay authority that no longer matches the declared replay semantics or the source/foundation bytes on disk, and a stale replay builder losing a publication race are all errors rather than silent reinterpretation. Missing required replay prediction state routes to prepare and is never rebuilt by a post-selection consumer.

Keyboard interruption returns the parser's documented interruption status and states that authenticated records remain resumable. Errors identify the owning command and the safe corrective action; they do not fall back to a different scientific selector or downstream model.

Acceptance boundary

The current CLI contract is established through the real parser/facade, generated-config parsing, current prepare/selection/CV/production owners, CampaignStore close/reopen, currentness reauthentication, storage cleanup owners, and exact-generation reject-before-reuse tests. The mandatory P5A6-to-P6 qualification additionally authenticates the baseline worktree and import roots before producing state, then opens the unchanged workspace in a separate P6 process.

P6 acceptance is current functional/restart/public-surface closure. It is not GPU, long real-data, M-ladder decision-preservation, deployment, physical, calibration, locked-test, or final-release qualification.